

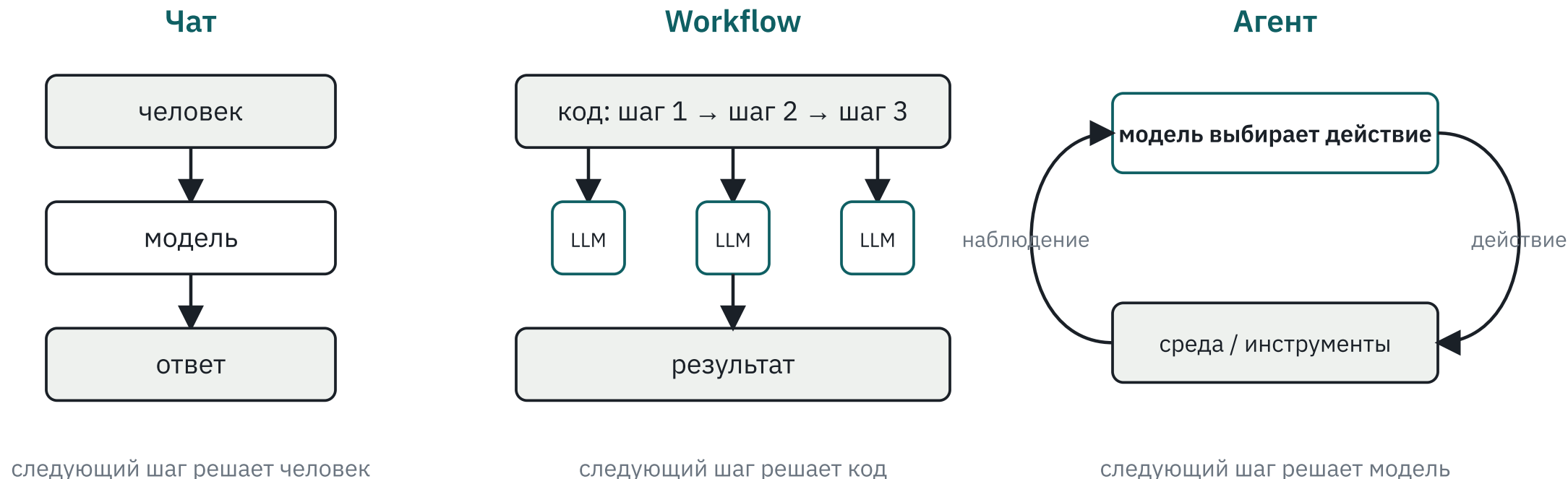
Лекция 1. Что такое агент

15 сентября 2026

О чём эта лекция

- Чем агент отличается от чата и от обычной программы с вызовами модели
- Как агент устроен внутри: цикл, инструменты, контекст
- Почему длинные цепочки шагов ломаются и зачем агенту проверки
- Сколько стоит один запуск агента и как платить меньше
- Из каких частей собирают агентные системы
- Демо: агент над базой объявлений о квартирах

От чата к агенту: кто решает следующий шаг



- **Чат:** модель отвечает один раз, дальше решает человек
- **Программа с LLM-шагами (workflow):** порядок шагов задан кодом
- **Агент:** модель сама решает, что делать дальше, по результатам предыдущих шагов

Так выглядит траектория: ReAct

(1d) ReAct (Reason + Act)

Thought 1: I need to search Apple Remote and find the program it was originally designed to interact with.
Act 1: Search[Apple Remote]
Obs 1: The Apple Remote is a remote control introduced in October 2005 by Apple ... originally designed to control the Front Row media center program ...

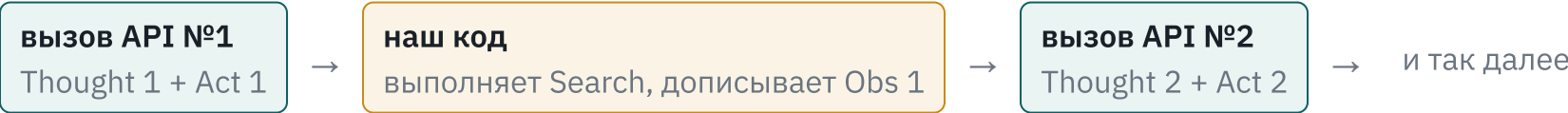
Thought 2: Apple Remote was originally designed to control the Front Row media center program. I need to search Front Row next and find what other device can control it.
Act 2: Search[Front Row]
Obs 2: Could not find [Front Row]. Similar: ['Front Row Seat to Earth', 'Front Row Motorsports', 'Front Row (software)', ...]

Thought 3: Front Row is not found. I need to search Front Row (software) .
Act 3: Search[Front Row (software)]
Obs 3: Front Row is a discontinued media center software ...

Thought 4: Front Row (software) is controlled by an Apple Remote or the keyboard function keys. So the answer is keyboard function keys.
Act 4: Finish[keyboard function keys]

- **Один шаг = один вызов API:** за одну генерацию модель пишет Thought N и Act N; цифра — номер шага, а не другая модель
- **Obs N** — не модель, а **результат от среды:** его дописывает наш код перед следующим **ВЫЗОВОМ**
- Повторяется, пока модель не скажет `Finish[...]`
- **Зелёным** авторы выделили цепочку фактов, ведущую к ответу: «Front Row media center» из Obs 1 → поиск в Act 2 → ответ в Act 4

Yao et al. ReAct, ICLR 2023 — фрагмент рис. 1 (панель 1d)



Как это выглядит на уровне API

ЗАПРОС: ЧТО МЫ ОТПРАВЛЯЕМ МОДЕЛИ

```
POST /v1/chat/completions
{
  "messages": [
    {
      "role": "user",
      "content": "Медианная цена м² в Ленинском?"
    }
  ],
  "tools": [
    {
      "type": "function",
      "function": {
        "name": "district_stats",
        "description": "Медианы цены и аренды",
        "parameters": {
          "type": "object",
          "properties": {
            "city": {
              "type": "string"
            },
            "district": {
              "type": "string"
            }
          }
        }
      }
    }
  ]
}
```

role — кто говорит: system (правила), user (задача), assistant (ответ), tool (результат)

content — текст сообщения; у ответа с вызовом он пустой

tools — доступные функции: имя, описание, JSON-схема аргументов. Это всё, что модель знает об инструменте

Ответ: модель **просит вызвать функцию**

```
{
  "choices": [
    {
      "message": {
        "role": "assistant",
        "content": null,
        "tool_calls": [
          {
            "id": "call_7f3",
            "function": {
              "name": "district_stats",
              "arguments": "{\n  \"city\": \"Ростов...\",\n  \"district\": \"Ленинский\"\n}"
            }
          }
        ]
      }
    }
  ],
  "usage": {
    "prompt_tokens": 412,
    "completion_tokens": 38
  }
}
```

Функцию выполняет **наш код** и шлёт результат обратно ролью tool

Один запуск агента: вход и выход

ЗАДАЧА	Найди три самые недооценённые двушки у метро и объясни почему
МОДЕЛЬ	вызов <code>search_listings(rooms=2, metro_dist_m≤1000)</code>
ИНСТРУМЕНТ	142 объявления (JSON)
МОДЕЛЬ	вызов <code>price_model_predict(ids=[...])</code>
ИНСТРУМЕНТ	оценки цены и остатки для 142 объектов
МОДЕЛЬ	три вызова в одном ответе: <code>get_listing(8812)</code> , <code>get_listing(8827)</code> , <code>get_listing(9004)</code> — прочитать описания лидеров
ИНСТРУМЕНТ	три результата, по одному на вызов: карточки с текстом
ОТВЕТ	Таблица из трёх объектов, причины недооценки, оговорки по рискам

- **Вход:** задача словами + доступ к данным через инструменты
- **Выход:** ответ и журнал всех шагов, и счётчик токенов
- Число шагов заранее неизвестно: его выбирает модель

Тот же запуск глазами модели

☐ уже было на предыдущем шаге ☐ добавилось на этом шаге

ШАГ 1	ВХОД МОДЕЛИ · 3 060 ТОКЕНОВ system prompt схемы инструментов задача	ВЫХОД МОДЕЛИ ВЫЗОВ search_listings(...)
ШАГ 2	ВХОД МОДЕЛИ · 9 400 ТОКЕНОВ system prompt схемы инструментов задача вызов 1 результат 1: 142 объявления	ВЫХОД МОДЕЛИ ВЫЗОВ price_model_predict(...)
ШАГ 3	ВХОД МОДЕЛИ · 12 600 ТОКЕНОВ system prompt схемы инструментов задача вызов 1 результат 1 вызов 2 результат 2: оценки цены	ВЫХОД МОДЕЛИ три вызова get_listing(...) параллельно
ШАГ 4	ВХОД МОДЕЛИ · 15 900 ТОКЕНОВ system prompt схемы инструментов задача вызов 1 результат 1 вызов 2 результат 2 вызовы 3 результаты 3: карточки	ВЫХОД МОДЕЛИ текст: таблица трёх объектов и объяснение

- Модель **не помнит** предыдущие шаги: каждый раз ей отправляют всю историю целиком
- Контекст растёт с каждым шагом — отсюда стоимость и ограничения на длину

Шаг 2 на уровне API

ЗАПРОС: ВСЯ ИСТОРИЯ В МАССИВЕ `messages`

```
POST /v1/chat/completions
{"model": "...",
 "messages": [
   {"role": "system",
    "content": "Ты аналитик рынка жилья..."},
   {"role": "user",
    "content": "Найди три недооценённые двушки..."},
   {"role": "assistant",
    "tool_calls": [{"id": "call_1",
                     "function": {"name": "search_listings",
                                   "arguments": "{\\"rooms\\": 2, ...}"}}],
   {"role": "tool", "tool_call_id": "call_1",
    "content": "[{\\"id\\": 8812, ...}, ... ещё 141]"},
 ],
 "tools": [ ...схемы 8 инструментов... ]}
```

system — правила: кто агент и чего не делает

user — задача пользователя; оба уходят на каждом шаге

assistant с `tool_calls` — просьба модели с шага 1, вернулась в историю

tool — результат от **нашего кода**, связан с вызовом по `tool_call_id`

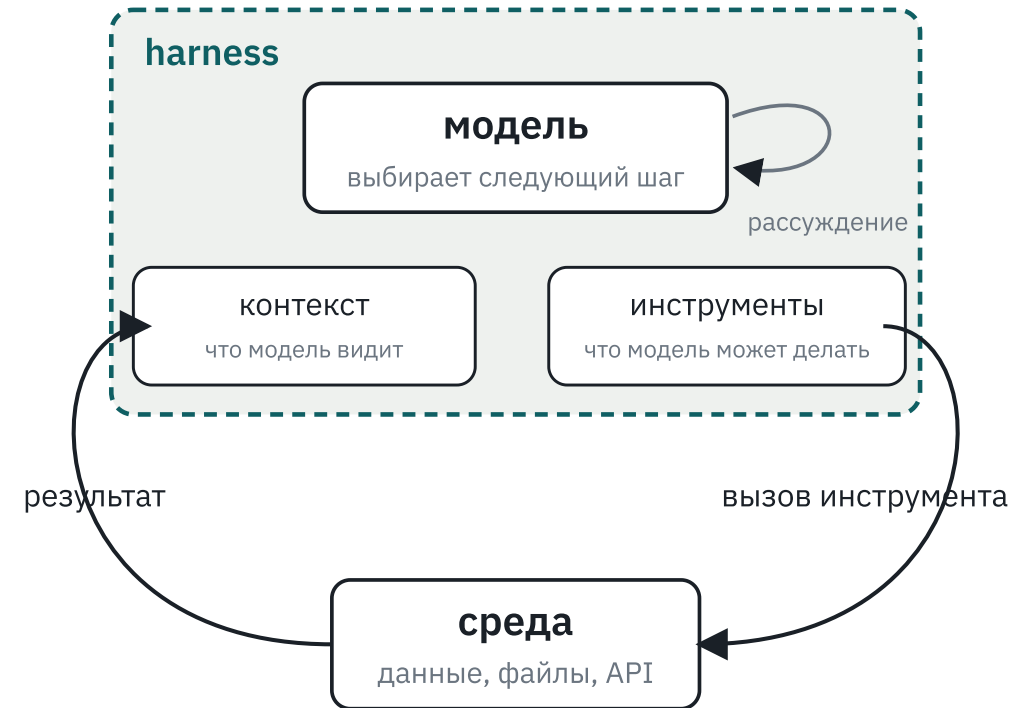
Ответ: следующий вызов инструмента

```
{"choices": [{"message": {
  "role": "assistant", "content": null,
  "tool_calls": [{"id": "call_2",
                   "function": {"name": "price_model_predict",
                                 "arguments": "{\\"ids\\": [8812, 8827, ...]"}
  }
}],
  "usage": {"prompt_tokens": 9400,
            "completion_tokens": 52}}
```

Откуда `ids`: модель взяла их из **результата первого вызова** — следующий шаг выбран по данным предыдущего, а не по плану

Словарь курса

- **Модель** — LLM: получает контекст, выдаёт текст или вызов инструмента
- **Инструмент** — функция, которую наш код выполняет по запросу модели
- **Контекст** — всё, что модель видит на этом шаге: инструкции, задача, история
- **Harness** (обвязка) — код вокруг модели: цикл, инструменты, сборка контекста, ограничения, журнал шагов
- Реализаций harness много: закрытые продукты (Codex, Claude Code, Cursor) и открытые (OpenHands, Aider, goose, smolagents); свой пишется за вечер



Кто отвечает за опасные действия

Агент правит файлы, запускает команды, отправляет письма. Кто решает, можно ли?

подход	КАК РАБОТАЕТ	ЧТО С НИМ НЕ ТАК
Спрашивать человека	каждое действие — «выполнить? у/н»	усталость от одобрений: люди жмут «да» не глядя
Автоматическая политика	правила или классификатор решают: выполнить, запретить, спросить	ошибки политики; редкие случаи всё же уходят человеку
Песочница	опасное невозможно физически: нет сети, нет записи вне папки	часть задач в песочнице не сделать

Пример: режимы разрешений в Claude Code

plan	только чтение и безопасные команды, правок нет	default	спрашивает при первом использовании каждого инструмента
acceptEdits	правки файлов в рабочей папке — без вопроса	auto	то, что классификатор счёл безопасным
bypassPermissions	всё; только в контейнере или виртуальной машине		

Политика разрешений в своём harness'e

Готовые режимы удобны, но в своём агенте это просто функция, которую harness зовёт **перед каждым вызовом инструмента**

```
def policy(call):                # решает harness, не модель
    if call.name in READ_ONLY:    # поиск, чтение, расчёт
        return "allow"
    if writes_outside(WORKDIR, call) or looks_destructive(call):
        return "deny"            # только то, что точно нельзя
    return "ask"                  # спорное — человеку, а не запрет
```

- **allow** — выполняем и возвращаем результат модели
- **deny** — не выполняем; модель получает отказ как обычное наблюдение и ищет другой путь
- **ask** — останавливаем цикл и показываем человеку вызов с аргументами
- Запрет по умолчанию превратил бы агента в бесполезного: `deny` ставят на заведомо опасное, остальное спорное уходит в `ask`

Настоящая политика смотрит на **аргументы**, а не только на имя инструмента: путь вне рабочей папки, адрес вне белого списка, команда, стирающая данные. А песочница остаётся страховкой на случай, когда политика ошиблась.

Почему агенты заработали именно сейчас

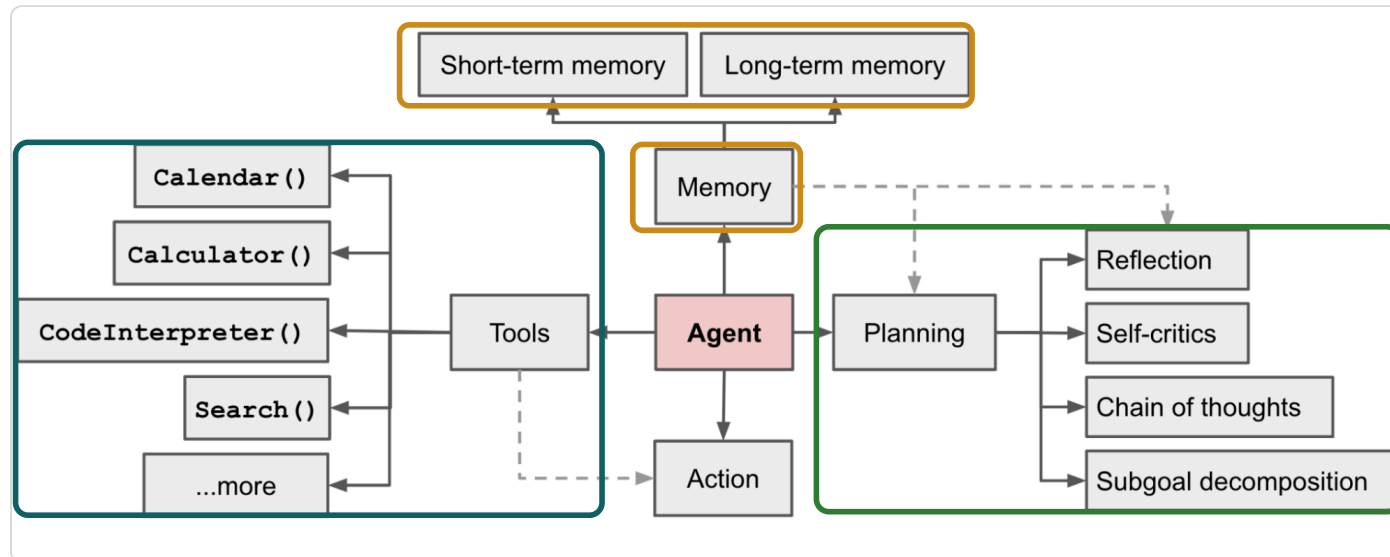
Идея «модель в цикле» старая. Работать она начала, когда сложились четыре вещи

2023	Вызов функций в API — модель возвращает имя и аргументы, а не текст «вызови поиск»; появились схемы инструментов в запросе
2023–2024	Контекст в 100 тысяч токенов и больше — вся история шагов помещается целиком, агент перестал забывать начало задачи
2024	Кеш префикса — повторяющаяся часть контекста стала стоить в разы дешевле, длинные циклы перестали разорять
2024–2025	Reasoning-модели — планирование между вызовами: модель обдумывает следующий шаг, а не только формулирует ответ
2025–2026	Агенты как продукты — готовые harness'ы для кода и работы с данными, общие протоколы для инструментов

- До этого агенты собирали из уговоров в промпте: «ответь строго в формате Action: ...» — и они разваливались на третьем шаге

Из чего состоит агент

Цикл, который мы разобрали, — это скелет. К нему добавляют три вещи, и из них складывается почти любой агент



Lilian Weng. LLM Powered Autonomous Agents, 2023

- **Инструменты** — чем агент действует; мы их уже видели в запросе
- **Память** — что остаётся между шагами и сессиями: история, заметки в файлах, база
- **Планирование** — как выбирается следующий шаг: рассуждение, разбиение задачи, самопроверка
- Модель в центре одна, всё вокруг строит инженер — это и есть предмет курса

Цикл агента как алгоритм

```
messages = [system_prompt, task]

for step in range(MAX_STEPS):
    reply = llm(messages, tools=TOOLS)
    messages.append(reply)

    if not reply.tool_calls:           # текст без вызовов
        return reply.content          # это финальный ответ

    for call in reply.tool_calls:
        result = run_tool(call)       # выполняет наш код
        messages.append(
            tool_message(call.id, result))

    tokens += reply.usage.prompt_tokens \
        + reply.usage.completion_tokens

raise StepLimitExceeded              # бюджет исчерпан
```

- Состояние агента — **список сообщений**
- Лимит шагов — страховка от бесконечного цикла
- Каждый шаг: отправляем **всю историю**, ответ дописываем в неё же
- Выход из цикла: модель ответила **текстом без вызовов**
- Инструменты выполняет **наш код**; результат — сообщением с ролью `tool`
- Токены — из поля `usage` каждого ответа
- Бюджет исчерпан — останавливаемся сами

Что ломается в цикле и что с этим делает harness

Цикл ломается в пяти предсказуемых местах

ЧТО ИДЁТ НЕ ТАК	КАК ЭТО ВЫГЛЯДИТ В ЖУРНАЛЕ	ЧТО ДЕЛАЕТ HARNESS
Не тот вызов	аргументы валидны по схеме, но район или инструмент выбран неверно	шаг проверки: сверить результат с условием задачи и вернуть модели расхождение
Ошибка инструмента	<code>TimeoutError</code> , пустая выборка, недоступная база	перехватить исключение и вернуть текст с подсказкой, что делать дальше
Зацикливание	тот же вызов с теми же аргументами третий раз подряд	детектор повторов: подсказать иную стратегию или остановить запуск
Переполнение контекста	инструмент вернул 2 000 объявлений — 130 000 токенов в истории	усечение результата, пагинация, сжатие старых шагов
Ложное «готово»	ответ «вот три лучшие квартиры», ни одна не сверена с данными	не принимать ответ без доказательства: цифры из инструмента или прогон проверки

- Ни одна из этих поломок не лечится промптом: это код вокруг модели

Проверка результата внутри цикла

Задача: *недооценённые двушки в Кировском районе*. Модель ошиблась в первом же вызове — дальше всё зависит от того, ловит ли эту ошибку наш код.

БЕЗ ПРОВЕРКИ

МОДЕЛЬ

search_listings(district="Ленинский")

в задаче был Кировский: модель перепутала район

ИНСТРУМЕНТ

96 объявлений — все из Ленинского

МОДЕЛЬ

price_model_predict(...)

ИНСТРУМЕНТ

оценки для 96 объектов

ОТВЕТ

«Три лучшие квартиры в Кировском»

ошибка

выборку никто не сверил с условием: ошибка дошла до ответа, и выглядит он уверенно

С ПРОВЕРКОЙ

МОДЕЛЬ

search_listings(district="Ленинский")

ИНСТРУМЕНТ

96 объявлений — все из Ленинского

ПРОВЕРКА

set(df.district) == {"Кировский"} → False

это обычный код в harness'е: поле выборки сравнили с условием задачи

НАБЛЮДЕНИЕ

«выборка из Ленинского, а нужен Кировский»

МОДЕЛЬ

search_listings(district="Кировский")

ОТВЕТ

три квартиры в Кировском, сверены с данными

ошибка вернулась в диалог как наблюдение — и модель исправила вызов сама

- **Проверка — это шаг цикла**, а не финальный аудит: тест, пересчёт по данным, сравнение с условием задачи
- Так устроены агенты для кода: правка → тесты → правка

Что из этого следует

- **Проверяемый шаг важнее умной модели.** Тест, пересчёт по данным, сравнение с условием задачи ловят ошибку там, где её ещё дёшево исправить
- **Повторить шаг — самый дешёвый ремонт.** Проверка не прошла — агент переделывает один шаг, а не весь запуск
- **Повтор шага и повтор запуска — разные вещи.** Первое чинит траекторию внутри одного запуска. Второе — способ померить агента: тот же вопрос прогоняют заново несколько раз и смотрят, всегда ли получается

Надёжность: один запуск ничего не доказывает

Один и тот же запрос даёт разные траектории: модель сэмплирует токены. Поэтому одну задачу прогоняют k раз в одинаковой среде.

	8 прогонов одной и той же задачи	хоть раз	все 8 раз
Задача А	✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓	да	да
Задача В	✓ ✓ ✗ ✓ ✓ ✗ ✓ ✓	да	нет
Задача С	✗ ✓ ✗ ✗ ✗ ✗ ✗ ✗	да	нет
итог по трём задачам:		3/3	1/3

$\text{pass}@8 = 1.00 \cdot \text{pass}^8 = 0.33$

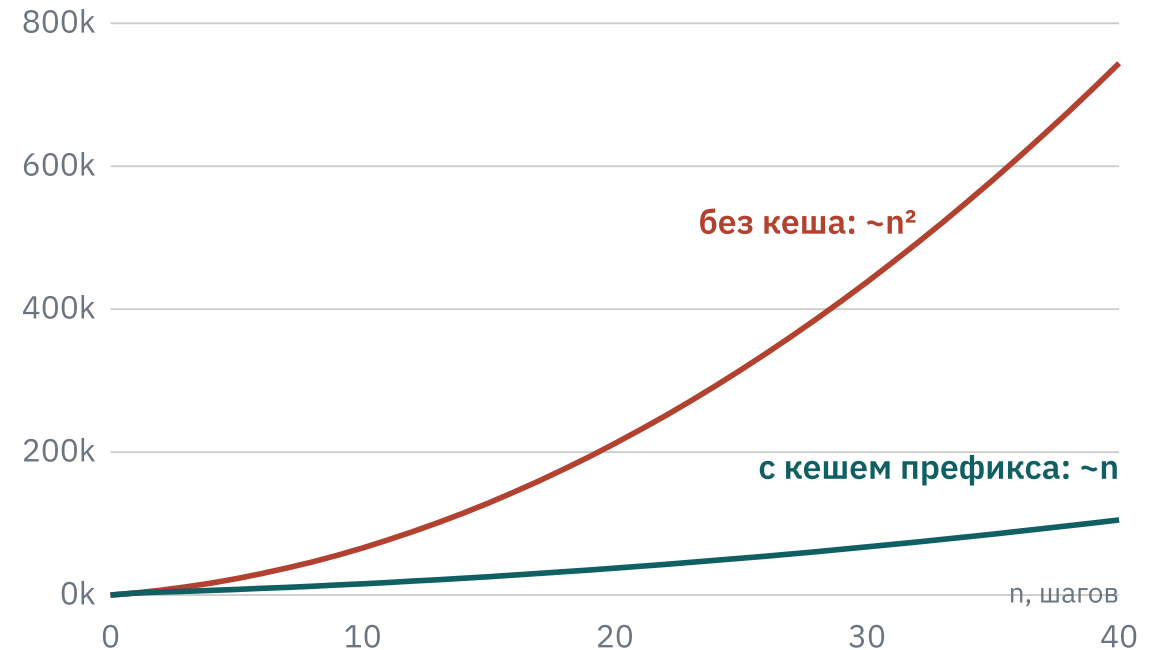
- $\text{pass}@k$, «pass at k» — получилось хотя бы раз из k : умеет ли модель это в принципе
- pass^k , «pass hat k» — получилось во всех k прогонах; вот это и есть надёжность
- Yao et al. τ -bench, 2024 (retail-домен): gpt-4o решает около 61% задач с одного прогона, pass^8 — ниже 25%

Сколько стоит один запуск агента

На каждом шаге модель читает **весь контекст**: инструкции c плюс t предыдущих шагов по r токенов

$$\sum_{t=0}^{n-1} (c + tr) = nc + \frac{r n(n-1)}{2}$$

- Без кеша входные токены растут **квадратично** по числу шагов
- С **кешем префикса** повторяющаяся часть стоит в разы дешевле — рост почти линейный
- Что именно повторяется — на следующем слайде



Кеш префикса: что именно повторяется

Контекст на **третьем** шаге агента:

system prompt · 1 200 ток.

схемы инструментов · 1 800

задача · 60

вызов 1 · 40

результат 1 · 900

вызов 2 · 40

результат 2 · 900

☐ уже было в предыдущем запросе — читается из кеша, в несколько раз дешевле ☐ новое — полная цена

- Кеш срабатывает по **совпадению начала** запроса байт в байт
- Поэтому история только **дописывается в конец**, а инструкции не меняются между шагами
- Одна дата-время в начале system prompt — и кеш не сработает ни разу
- Провайдер сообщает число кешированных токенов в `usage`

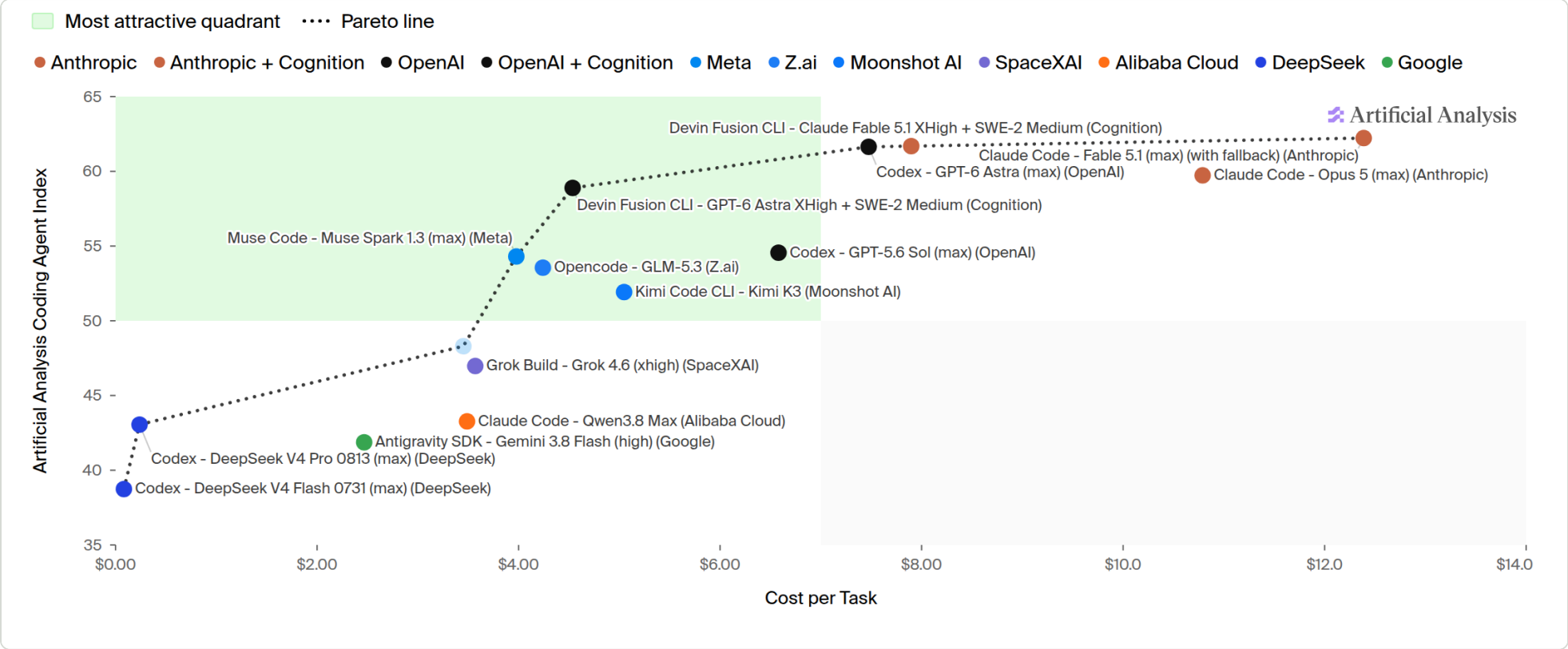
Пример с числами

$c = 3000$ токенов инструкций, $r = 800$ токенов на шаг, $n = 20$ шагов

	ВХОДНЫХ ТОКЕНОВ ЗА ЗАПУСК	ПО \$0.20 ЗА 1М	1000 ЗАПУСКОВ В ДЕНЬ
Без кеша	212 000	\$0.042	\$42
С кешем префикса	≈ 37 580	\$0.008	\$8

- Разница почти в шесть раз при тех же действиях агента
- Деньги считаются в лоб: токены ÷ 1 000 000 × ставка за миллион
- Кешированные токены стоят 10% от обычных: сентябрь 2026, у Gemini 2.5 Flash это 0.30 против 0.03 доллара за 1М, у младшей модели OpenAI — 0.20 против 0.02
- Посчитайте то же по журналу своего агента: поля `usage` есть в каждом ответе

По чему сравнивают агентов



artificialanalysis.ai/agents/coding-agents, снимок 14 сентября 2026 — индекс по трём агентным бенчмаркам против средней цены задачи

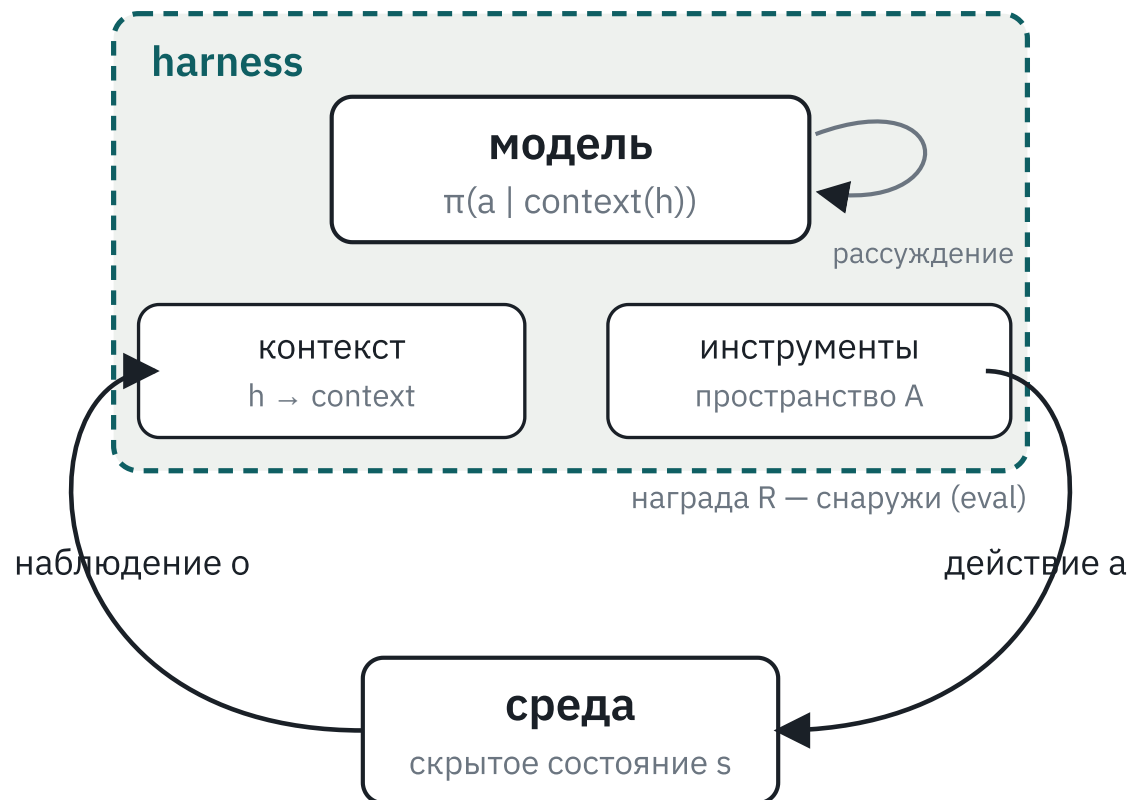
- Оси сравнения: **качество, стоимость за задачу, скорость**
- Точки — связки **обязка + модель**: та же модель в другой обязательке даёт другой результат
- Слева вверху дёшево и сильно; правее платят в разы больше за пару пунктов индекса

Из чего собирают агентную систему



- Над водой видна модель, но основной объём системы — это обвязка: цикл, инструменты, сборка контекста, разрешения, журнал
- Слои независимы: модель меняется, не трогая остальное; на одном фреймворке собирают совсем разные агенты
- В готовых продуктах — Codex CLI, Claude Code, Cursor — обвязка это тысячи строк обычного кода, написанного людьми

Агент и среда на языке RL



Что здесь можно оптимизировать

- **Harness** — наш код: инструменты, сборка контекста, проверки. Правим руками, эффект виден сразу
- **Веса модели** — та самая политика π . Меняются только обучением: SFT на удачных траекториях, дальше RL по награде
- Обучение не трогает harness, но целиком зависит от него: обвязка задаёт среду, список действий и ту проверку, из которой берётся награда
- **Рассуждение — внутреннее действие:** среду оно не меняет, действий среды всего два вида — вызов инструмента и финальный ответ. Но политика порождает рассуждение теми же токенами, и при обучении градиент идёт и по ним

Дообучать агента или нет

Стоит, когда сходятся все четыре

1. Результат **проверяется программой**: тест, сверка с эталоном, пересчёт — из этого и делается награда
2. Задача **узкая и повторяется** тысячи раз: есть ради чего менять дорогую модель на дешёвую открытую
3. Промпт, инструменты и контекст **уже вычерпаны**, надёжность упёрлась
4. Есть **траектории**: свои журналы запусков или открытые данные

Иначе выигрыш даёт обвязка, а не веса

Что получилось, когда сходилось

- **DeepSeek-R1** (2025). Задача: математика, код, STEM — ответ сверяется автоматически. Чистый RL без размеченных рассуждений: самопроверка возникает сама
- **WebRL** (2024). Задача: выполнять поручения на сайтах — магазин, форум, GitLab (WebArena-Lite). Открытая Llama-3.1-8B: с 4.8% до 42.4%, выше GPT-4-Turbo с 17.6%
- **SWE-RL** (2025). Задача: чинить настоящие issue в GitHub-репозиториях, награда — сходство патча с эталонным. 41% SWE-bench Verified, лучший тогда результат среди моделей меньше 100B

Что мы разобрали

- **Определение агента** — чем он отличается от чата и от программы с LLM-шагами
- **Уровень API** — `tools` в запросе, `tool_calls` в ответе, роль `tool`, где считаются токены
- **Harness** — цикл, инструменты, сборка контекста, разрешения, бюджеты, журнал
- **Поломки цикла** — пять типовых и почему проверка должна быть шагом цикла
- **Метрики надёжности** — $\text{pass}@k$ и pass^k : зачем одну задачу гоняют k раз
- **Стоимость запуска** — квадратичный рост входа и кеш префикса
- **Оси сравнения** — качество, стоимость, скорость; сравниваем связки «обязка + модель»
- **Язык RL** — что оптимизируют, когда дообучение оправдано и когда хватает обвязки

Как устроен курс

- **Проект** — один на семестр: тема согласуется с преподавателем, сдаётся и защищается в декабре
- **Практические задания** — пять штук, независимых от проекта; для зачёта нужно минимум четыре
- **Доклад по статье** можно сдать вместо одного из заданий: 20–30 минут на паре
- **Экзамен** в январе по всей программе; при отличном проекте и всех пяти заданиях — автомат
- **Работаем на своей машине**: Python и VS Code; агент для кода на выбор — Claude Code, Codex или другой
- **Фреймворк** — Google ADK или OpenAI Agents SDK, курс построен на них; любой другой разрешён под свою ответственность
- Подписки на сервисы понадобятся: ключи к моделям, агент для кода, аренда GPU

Дополнительно: о чём можно делать проект

Подойдёт любая задача, где агент делает несколько шагов, а результат можно проверить не «на глаз» 😊

1. 🏠 **Инвест-аналитик недвижимости** — по моим параметрам ищет объекты дешевле рынка: модель цены, доходность аренды, ипотечные сценарии
2. 📈 **Бэкестинг стратегий** — гипотеза словами → код стратегии → бэкест на открытых котировках → метрики и проверка на переобучение
3. 📖 **Ассистент исследователя** — собирает статьи по теме, хранит, размечает, ищет по ним и отвечает с точными цитатами
4. 📁 **Карьерный агент** — вакансии под профиль через открытый API hh.ru, разрыв в навыках, статистика зарплат, письма работодателям и отслеживание откликов
5. 💳 **Личные финансы по выпискам** — категории трат, подписки, аномалии, план бюджета на своих данных
6. 🧩 **Агент-доказатель на Lean 4** — формализует утверждение, ищет леммы в Mathlib, итерирует по ошибкам компилятора
7. 💡 **Своя тема** — реальный пользователь, несколько инструментов, проверяемый результат

Как выбирать тему

- Результат **проверяется программно** хотя бы частично: тест, пересчёт, компилятор, разметка
- Нужны **несколько инструментов и несколько шагов** — иначе это не агент, а один вызов модели
- **Данные доступны**: открытые наборы, официальные API, свои данные, синтетика. Парсинг сайтов — серая зона, на свой страх и риск; готовые сборщики вроде Apify снимают техническую часть, но не правовую
- Данных достаточно, чтобы **держат их в базе**: SQL, Elasticsearch или векторный индекс, а поиск по ним оформлен как инструмент агента
- Есть **пользователь**, пусть даже вы сами, и понятно, что для него «хорошо»
- Есть **опасное или необратимое действие** — тогда есть о чём думать в части безопасности

Доклады по статьям

Что это. Одна статья, разбор на 20–30 минут. Засчитывается вместо одного практического задания.

Как построить рассказ

- Какую задачу решали и почему она была нерешённой
- Что предложили: метод в одну схему
- Что получилось: главный график и ограничения
- Что из этого взять в свой проект

С чего начать выбор  — статьи 2026 года

- Steer, Don't Solve — критик для код-агента
- Evolve with Their Harness — меняющаяся обвязка
- Structured Context Engineering — формат контекста
- Escaping the Context Bottleneck — контекст через RL
- Memory for Autonomous LLM Agents — обзор памяти
- WideSeek-R1 — мультиагентный RL
- Malicious Agent Skills — вредоносные скиллы
- Show-Harness — VLM-агент для роботов и GUI
- Классика и своя тема — по согласованию

Что сделать за неделю

- Поставить **Python** с **uv** и **VS Code** по инструкции курса
- Подключить **агента для кода**: Claude Code, Codex или другой; проверить, что он видит проект и умеет запускать команды
- Получить **доступ к моделям**: ключ Gemini или OpenAI, сразу поставить лимит расходов
- Написать с агентом **две-три маленькие программы** и убедиться, что они работают: запустить, прочитать код, поправить
- Попробовать **вайб-кодинг**: сформулировать задачу словами, дать агенту сделать целиком, потом проверить результат самому
- Записать, **где агент ошибся** и что пришлось править руками: на практике это первое, что обсудим

Дополнительное чтение

- Lilian Weng «LLM Powered Autonomous Agents» (2023) — lilianweng.github.io
- Anthropic «Building effective agents» (2024) — граница между программой с LLM-шагами и агентом
- Sumers et al. «Cognitive Architectures for Language Agents» (2024), разделы 1–3 — arxiv.org/abs/2309.02427
- Yao et al. «ReAct» (ICLR 2023) — arxiv.org/abs/2210.03629
- Kapoor et al. «AI Agents That Matter» (2024) — arxiv.org/abs/2407.01502
- Chip Huyen, *AI Engineering* (2025), глава про агентов